1. Defining Constructor

Definition:

A constructor is a special member function with the same name as the class, automatically called when an object is created. It initialises the object's data members. No return type (not even `void`).

Very Basic Example:

```
class Box {
public:
    int length;
    // Constructor
    Box() {
        length = 0;
        cout << "Box created" << endl;
    }
};

int main() {
    Box b;          // constructor called
    cout << b.length; // 0
    return 0;
}
```

Practice Question:

Create a class `Circle` with a constructor that initialises radius to 5.0 and prints `"Circle created"`. Instantiate an object in `main()` and display its radius.

2. Overloaded Constructors

Definition:

A class can have multiple constructors with different parameter lists (number or types of parameters). The compiler chooses the appropriate constructor based on the arguments passed during object creation.

Very Basic Example:

```
class Rectangle {
    int width, height;
public:
    Rectangle() { width = height = 0; }          // default
    Rectangle(int w, int h) { width = w; height = h; } // parameterised
};

int main() {
    Rectangle r1;       // default
    Rectangle r2(5, 10); // parameterised
}
```

Practice Question:

Write a class `Student` with two constructors: one default that sets `name` to `"Unknown"` and `age` to 0, and another that takes a name and age. Create both types of objects and display their data.

3. The Default Copy Constructor

Definition:

If you don't provide a copy constructor, C++ supplies a default one that performs a **member-wise copy** (shallow copy). It's invoked when a new object is created as a copy of an existing object.

Very Basic Example:

```
class Point {
public:
    int x, y;
    Point(int a, int b) { x = a; y = b; }
    // no copy constructor defined – compiler provides default
};


int main() {
    Point p1(3, 4);
    Point p2 = p1;   // default copy constructor called
    cout << p2.x << " " << p2.y; // 3 4
}
```

Practice Question:

Create a class `Car` with attributes `model` (string) and `year` (int) and a parameterised constructor. Copy one object to another using the default copy constructor. Print both objects to verify the copy. What limitation might this shallow copy have if a pointer member existed?

4. Defining Destructors

Definition:

A destructor has the same name as the class preceded by `~`, takes no arguments, and is called automatically when an object goes out of scope or is `deleted`. It is used for cleanup (e.g., releasing memory, closing files).

Very Basic Example:

```
class Sample {
public:
    Sample()  { cout << "Constructor\n"; }
    ~Sample() { cout << "Destructor\n"; }
};

int main() {
    Sample obj; // constructor called
    // when main ends, destructor called automatically
}
```

Practice Question:

Define a class `Logger` that prints `"Log started"` in its constructor and `"Log ended"` in its destructor. Create two objects inside a block `{}` and observe the order of destruction messages.

5. Destructor Example (Resource Cleanup)

Definition:

A practical use of a destructor is to free dynamically allocated memory or resources acquired during the object's lifetime, preventing memory leaks.

Very Basic Example:

```
class DynamicArray {
    int* arr;
public:
    DynamicArray(int size) {
        arr = new int[size];
        cout << "Memory allocated\n";
    }
    ~DynamicArray() {
        delete[] arr;
        cout << "Memory freed\n";
    }
};

int main() {
    DynamicArray da(10);
    // destructor automatically frees memory
}
```

Practice Question:

Modify the DynamicArray class to accept values, store them, and display them. Add a member function to print the array. Ensure that the destructor correctly deallocates memory. Test with a small program.

6. Static Class Data (Static Members)

Definition:

A `static` data member belongs to the class as a whole, not to any particular object. Only one copy exists, shared by all objects. It must be defined outside the class.

Very Basic Example:

```
class Employee {
public:
    static int count;   // declaration
    Employee() { count++; }
};
int Employee::count = 0; // definition

int main() {
    Employee e1, e2, e3;
    cout << Employee::count; // 3
}
```

Practice Question:

Create a class `Student` with a static data member `totalStudents` that increments in the constructor and decrements in the destructor. In `main()`, create a block and instantiate a few objects, then print `totalStudents` before and after the block.

7. Introduction to Inheritance

Definition:

Inheritance allows a class (derived class) to acquire properties and behaviours (data members and functions) of another class (base class). It promotes code reuse and hierarchical classification.

Very Basic Example:

```
class Animal {   // base class
public:
    void eat() { cout << "Eating\n"; }
};

class Dog : public Animal { // derived class
public:
    void bark() { cout << "Barking\n"; }
};

int main() {
    Dog d;
    d.eat();   // inherited
    d.bark();
}
```

Practice Question:

Define a base class `Vehicle` with a method `start()` that prints `"Vehicle started"`. Derive a class `Car` publicly and add a method `honk()`. Create a `Car` object and call both methods.

8. Derived Class and Base Class

Definition:

The **base class** is the class being inherited from; the **derived class** is the class that inherits. The derived class can add new members and override existing ones.

Very Basic Example:

```
class Shape {       // base
public:
    void setWidth(int w) { width = w; }
protected:
    int width;
};
class Rectangle : public Shape { // derived
public:
    int getArea() { return width * height; }
    void setHeight(int h) { height = h; }
private:
    int height;
};
```

Practice Question:

Create a base class `Person` with `name` and `age`. Derive a class `Teacher` that adds `subject`. Write a member function in `Teacher` to display all details. Test it in `main()`.

9. Accessing Base Class Members

Definition:

A derived class can access **public** and **protected** members of the base class directly. Private members of the base class are not accessible directly in the derived class, but can be accessed through public/protected member functions of the base.

Very Basic Example:

```
class Base {
public:
    int pub;
protected:
    int prot;
private:
    int priv;
};

class Derived : public Base {
public:
    void show() {
        pub = 1;   // OK
        prot = 2;  // OK
        // priv = 3; // ERROR – not accessible
    }
};
```

Practice Question:

Design a base class `Account` with `protected` balance and a `public` `deposit()` method. Derive `SavingsAccount`. Write a function in `SavingsAccount` that accesses `balance` and `deposit()`. Show what happens if you try to access a `private` member directly.

10. The protected Access Specifier

Definition:

protected members are similar to private, but they are accessible within the class itself and by derived classes. They are not accessible from outside the class hierarchy (i.e., not by unrelated code).

Very Basic Example:

```
class Base {
protected:
    int secret;
};

class Derived : public Base {
public:
    void reveal() {
        secret = 42;      // accessible
        cout << secret;
    }
};

int main() {
    Derived d;
    // d.secret = 10;     // ERROR – protected
    d.reveal();
}
```

Practice Question:

Create a class BankAccount with a protected data member accountNumber. Derive PremiumAccount. In main(), try to access accountNumber directly from a PremiumAccount object (should fail) and write a public member function to display it (should succeed).

11. Derived Class Constructors

Definition:

When creating a derived class object, the base class constructor is called first (to initialise the base part), then the derived constructor runs. The derived constructor can explicitly call a specific base constructor in its initialiser list; otherwise the default base constructor is called.

Very Basic Example:

```
class Parent {
public:
    Parent() { cout << "Parent default\n"; }
    Parent(int x) { cout << "Parent param: " << x << "\n"; }
};

class Child : public Parent {
public:
    Child() { cout << "Child default\n"; }
    Child(int a) : Parent(a) { cout << "Child param\n"; }
};

int main() {
    Child c1;     // Parent default, Child default
    Child c2(5);  // Parent param: 5, Child param
}
```

Practice Question:

Write a base class `Building` with a constructor that takes `floors` (int) and prints it. Derive `House` that adds `rooms` and has a constructor taking both `floors` and `rooms`. Use the initialiser list to call the base constructor. Test with an object.

12. Base Class Constructors (Calling from Derived)

Definition:

A derived class can (and often must) invoke a specific base class constructor from its member initialiser list. This ensures the base portion is properly initialised before the derived part. If not called, the default base constructor is used.

Very Basic Example (combined with previous):

```
class Animal {
public:
    Animal(string name) { cout << "Animal: " << name << "\n"; }
};

class Dog : public Animal {
public:
    Dog(string breed) : Animal("Dog") {
        cout << "Breed: " << breed << "\n";
    }
};

int main() {
    Dog d("Labrador");
    // Output: Animal: Dog
    //         Breed: Labrador
}
```

Practice Question:

Design a class `Employee` with a constructor that accepts `id` and `name`. Derive `Manager` that also has `department`. The `Manager` constructor should accept all three and explicitly call the base constructor. Write a `display()` method and test.

13. Public Inheritance

Definition:

When deriving with `public` inheritance, public members of the base class remain public in the derived class, protected members remain protected, and private members are inaccessible directly. This is the most common type of inheritance.

Very Basic Example:

```
class Base {
public:    int a;
protected: int b;
private:   int c;
};

class Derived : public Base {
public:
    void access() {
        a = 1; // OK, public
        b = 2; // OK, protected
        // c = 3; // ERROR
    }
};

int main() {
    Derived d;
    d.a = 10; // OK – still public
    // d.b = 20; // ERROR
}
```

Practice Question:

Create a class `Device` with `public` `brand` and `protected` `serial`.

Derive `Laptop` publicly. Write a `main()` that accesses `brand` directly and also demonstrates that `serial` is inaccessible directly but can be read through a public method.

---

14. Protected Inheritance

Definition:
With `protected` inheritance, both public and protected members of the base class become **protected** in the derived class. They are accessible inside the derived class and its further derived classes, but not from the outside.

Very Basic Example:

```
class Base {
public:    int a;
protected: int b;
};

class Derived : protected Base {
public:
    void access() {
        a = 1; // OK (now protected)
        b = 2; // OK
    }
};

int main() {
    Derived d;
    // d.a = 10; // ERROR – a is protected
    d.access();
```

}

Practice Question:

Given a base class `Tool` with `public` method `use()`,
derive `Hammer` with `protected` inheritance. Try to call `use()` on a `Hammer` object in `main()` (should fail). Then create a public method in `Hammer` that internally calls `use()`. Explain the access changes.

---

15. Private Inheritance

Definition:

When deriving with `private` inheritance, all public and protected members of the base class become **private** in the derived class. They can only be used within the derived class; further derived classes cannot access them.

Very Basic Example:

```
class Base {
public:    void show() { cout << "Base\n"; }
protected: int x;
};

class Derived : private Base {
public:
    void display() {
        show(); // OK, now private member of Derived
        x = 5;  // OK
    }
};

class MoreDerived : public Derived {
```

```
public:
    void tryAccess() {
        // show(); // ERROR – Base members are private in Derived
    }
};

int main() {
    Derived d;
    // d.show(); // ERROR – private
    d.display();
}
```

Practice Question:

Write a class Engine with a public method start(). Derive Car using private inheritance. Create a main() that tries to call start() on a Car object (error). Then add a public method ignite() in Car that calls start(). Verify that a further derived class (e.g., SportsCar from Car) cannot access start() at all.